Planning and Approximate Reasoning

Autonomous Drone Planning for Smart-City Emergency Response

Professor: Dr. Najeeb Jebreel

Students: Brénedin Enrique Gómez Núñez

&

Fardin Ahmed

Date of submission: 05/11/2026

Table of contents

Table of contents 2
Introduction 3
Problem analysis 3
Types of objects 3
Predicates 3
Actions 4
PDDL implementation 4
Alternative Models (Model A vs. Model B) 7
Recharge Mechanics 7
Search Complexity and Plan Quality 7
Trade-off Summary 8
Problem instances 8
Instance 1 8
Instance 2 8
Instance 3 9
Planner setup 10
Results and analysis 11
Problem-small analysis 11
Problem-medium analysis 12
Problem-large analysis 12
Problem-large with only 1 Drone analysis 13
Literature Connection 13
Limitations and future work 14
Appendix 15
References 61

Introduction

Problem analysis

Search Space

We model a situation where one or more drones must complete inspection and supply-delivery tasks while minimizing the total mission cost. The planner must consider different combinations of drone positions, battery levels, supply availability, completed tasks, and site accessibility when deciding which actions to perform. Each action changes the current state of the environment and affects which actions are possible next.

For our small problem instance, we can approximate the space search size by looking at all the possible states our objects can be at.

$$|\text{States}| \approx (L \times B) \times 2^I \times S$$

where L is the number of locations, B the number of battery levels, I the number of inspection goals (a 2 is chosen since locations can either be inspected already or not), and S the number of possible supply states.

Substituting the values of Instance 1:

$$|\text{States}| \approx (5 \times 5) \times 2^2 \times 3 = 300$$

This describes the combinatorial nature of planning and how the space search grows exponentially as more locations and objects that can perform actions are added, the combination of state variables (objects and predicates) also grows.

Types of objects

Type	Description
drone	Autonomous vehicles executing inspections and deliveries.
location	Depots, mission sites, charging stations in the city model.
supply	Emergency resources transported by the drones, their subtypes are med kits, water, or communications equipment (comms).
level	Symbolic battery levels used to represent battery capacity and consumption.

Predicates

The domain predicates are organized into five main groups:

- **DRONE RELATED:** Tracks drone locations, carried supplies, and battery levels.

- **LOCATION RELATED:** Defines flight paths, whether mission sites are accessible or their unlocking requirements.
- **SUPPLY RELATED:** Tracks package locations, delivery requirements, and completed deliveries.
- **CHARGING RELATED:** Identifies locations where drones can recharge.
- **BATTERY RELATED:** Represents battery usage during flights and charging.

Actions

The PDDL domain defines five main actions that allow drones to complete their missions. **Fly** moves a drone between locations while consuming battery according to the distance travelled. **Inspect** allows a drone to inspect a location and potentially unlock other locations. **Pick-up-supply** allows a drone to collect a package for later delivery, while **Deliver-supply** allows it to deliver the package to a location where it is required. Finally, **Recharge** allows a drone to restore its battery at a charging station. The **main difference** between the two models is the recharge behavior: Model A restores the battery to its maximum level in one action, while Model B increases it by one level at a time.

We create a numeric variable called **(total-cost) – number** to use as our metric for optimization A function called **(distance ?from ?to)** is also created to get the numeric value of every possible flight trajectory between nodes.

We then say that every action has an increase of 1, **(increase (total-cost) 1)**, except for the fly action, which increases the value with the distance between each site **(increase (total-cost) (distance ?from ?to))**.

Design decisions

One of the main design decisions was how to represent battery levels in the PDDL domain. Since Fast Downward does not support the numeric operations needed to directly increase or decrease battery values, we represented each battery level as a separate object (b0, b1, b2, etc.). We then used “lookup facts” in the problem files to define how battery levels change during flights and recharging. This allowed us to model battery consumption and charging using standard PDDL predicates while keeping the two recharge models separate.

PDDL implementation

3.1 State Representation (Predicates)

State space is defined by 17 predicates that can be categorized into groups related to the object or process they describe.

Functional group	Predicates	Description
LOCATION RELATED	(at ?d - drone ?l - location)	Tracks the current location of a drone.
	(inspected ?l - location)	The location has been inspected.
	(unlocked-by-inspection ?source ?target - location)	Dependency: inspecting a source location unlocks a target location.
	(unlocked-by-delivery ?source ?target - location)	Dependency: delivering a supply to a source location unlocks a target location.
	(connected ?from ?to - location)	Which sites are interconnected, it's a two ways relation.
	(accessible ?l - location)	The location is unlocked and can be accessed.
DRONE RELATED	(empty ?d - drone)	True if drone is carrying nothing.
	(carrying ?d - drone ?s - supply)	What supply is the drone carrying.
	(battery ?d - drone ?l - level)	How much battery the drone has.
BATTERY RELATED	(flight-cost ?from ?to - location ?c - level)	flying from ?from to ?to costs ?c battery levels.
	(battery-after ?before ?c ?left - level)	subtracting ?c battery level from ?before.
	(next-level ?l1 ?l2 - level)	level ?l2 is exactly one unit higher than ?l1.
	(max-level ?l - level)	maximum battery level for each drone.
CHARGING RELATED	(has-charging-station ?l - location)	Identifies locations where drones can recharge.
SUPPLY RELATED	(supply-at ?s - supply ?l - location)	Tracks the location of a supply.
	(needs ?l - location ?s - supply)	the location ?s needs a specific supply.
	(delivered ?s - supply ?l - location)	the required supply was delivered.

Figure 1. Predicates of our world representation.

3.2 Actions

The domain implements five actions. Each action manages the preconditions of the environment while updating the numeric (total-cost) function for plan optimization.

1. Fly

(:action fly

```
   :parameters (?d - drone ?from ?to - location
               ?before ?c ?left - level)

   :precondition (and
      (at ?d ?from)                  ; Drone is at the starting location
      (connected ?from ?to)          ; A flight path exists
      (accessible ?to)               ; Destination is accessible
      (battery ?d ?before)           ; Current battery level
      (flight-cost ?from ?to ?c)     ; Battery cost of the flight
      (battery-after ?before ?c ?left) ; Remaining battery after the flight
   )
   :effect (and
      (not (at ?d ?from))            ; Remove the previous location
      (at ?d ?to)                    ; Move drone to the destination
      (not (battery ?d ?before))     ; Remove the previous battery level
      (battery ?d ?left)             ; Set the new battery level
      (increase (total-cost) (distance ?from ?to)) ; Add travel distance to total cost
   )
)
```

2. Inspect

(:action inspect

```
   :parameters (?d - drone ?l - location)
   :precondition (and
      (at ?d ?l)                      ; Drone must be physically present at the target
location
      (not (inspected ?l))           ; Location must not have been inspected yet
```

```
   )
   :effect (and
      (inspected ?l)                ; Mark the current location as inspected
      (forall (?y - location)       ; Iterate over all possible map locations
        (when (unlocked-by-inspection ?l ?y)
        (accessible ?y)           ; Grant accessibility to dependent locations unlocked
by this inspection
        )
      )
      (increase (total-cost) 1)      ;every action action increases the total-cost by 1
   )
)
```

3. pick-up-supply

```
(:action pick-up-supply
  :parameters (?d - drone ?s - supply ?l - location)
  :precondition (and
      (at ?d ?l)
      (supply-at ?s ?l)           ; The target supply must physically reside at this
location
      (empty ?d)                    ; The drone must currently be free (not carrying
anything)
  )
  :effect (and
      (not (supply-at ?s ?l))    ; Remove the supply from the location (it is now picked
up)
      (not (empty ?d))           ; Mark the drone as no longer empty
      (carrying ?d ?s)           ; Bind the specific supply to the drone's inventory
      (increase (total-cost) 1)  ;every action action increases the total-cost by 1
   )
)
```

4. deliver-supply

```
(:action deliver-supply
  :parameters (?d - drone ?s - supply ?l - location)
  :precondition (and
      (at ?d ?l)
      (carrying ?d ?s)           ; Drone must currently be carrying the specific supply
      (needs ?l ?s)              ; The target location must require this supply
  )
  :effect (and
      (not (carrying ?d ?s))
      (empty ?d)
      (delivered ?s ?l)            ; Mark the supply as successfully delivered to this
location
      (supply-at ?s ?l)         ; Register the supply physically present at the location
      (not (needs ?l ?s))         ; the site does not need the supply anymore
      (forall (?y - location)    ;For all locations on the problem
        (when (unlocked-by-delivery ?l ?y) ; search for the ones where unlocked by
delivery is true
          (accessible ?y)        ; And unlock the location
        )
      )
      (increase (total-cost) 1)  ;every action action increases the total-cost by 1
   )
)
```

5.1 recharge (Model A)

```
(:action recharge
   :parameters (?d - drone ?l - location ?cur ?max - level)
   :precondition (and
```

```
      (at ?d ?l)                  ; Drone must be located at the charging station
      (has-charging-station ?l)   ; Location must have a charging station
      (battery ?d ?cur)           ; Drone must have its current battery level registered
    (max-level ?max)            ; References the maximum battery level
   )
   :effect (and
      (not (battery ?d ?cur))     ; Clear the current partial battery level
      (battery ?d ?max)           ; Instantly set battery to the maximum capacity level
      (increase (total-cost) 1)   ;every action action increases the total-cost by 1
   )
)
```

5.2 recharge (Model B)

```
(:action recharge
   :parameters (?d - drone ?l - location ?cur ?next - level)
   :precondition (and
      (at ?d ?l)                  ; Drone must be located at the charging station
      (has-charging-station ?l)   ; The location must feature an active charging station
      (battery ?d ?cur)           ; Drone must have its current battery level registered
      (next-level ?cur ?next)     ; Defines the next incremental battery step upwards
   )
   :effect (and
      (not (battery ?d ?cur))     ; Clear the current lower battery level
      (battery ?d ?next)          ; Update the drone to the next higher battery level
      (increase (total-cost) 1) ;every action action increases the total-cost by 1
   ))
```

Alternative Models (Model A vs. Model B)

Both models represent the emergency drone scenario, but they use different recharging methods. This affects how the planner searches for a solution and the length of the plans it produces.

Recharge Mechanics

- Model A (Instant Full Charge): A single recharge action restores the drone's battery to its maximum level. This simplifies real-world charging, as the drone does not need to wait for each battery level. It also keeps the planning process simpler.
- Model B (Incremental Charge): Each recharge action increases the battery by one level using the next-level predicate. This is more realistic because it represents charging step by step. It also allows drones to recharge only as much as needed for their next task.

Search Complexity and Plan Quality

- Search Effort: Model A completes recharging in one action, while Model B requires the planner to consider each intermediate battery level. On the Medium instance, A* (LM-cut) expanded 446,517 states for Model B and took 21.83 seconds. Model A expanded 184,237 states and took 13.05 seconds.
- Plan Metrics: Model B produces longer plans because it may require several recharge actions. On the Medium instance, Model A produced a 28-step plan with a cost of 32, while Model B produced a 34-step plan with a cost of 38.
- Scalability: Both models timed out on the large instance with three drones when using A* (LM-cut). However, on the large instance with one drone, Model A solved the problem in

0.42 seconds after expanding 4,614 states. Model B took 0.56 seconds and expanded 7,528 states.

Trade-off Summary

Model A is simpler and generally requires less search effort, but it does not represent charging in much detail. Model B provides a more realistic charging process, but its additional battery levels increase the number of states the planner needs to consider. Therefore, Model A is more efficient in the tested instances, while Model B provides a more detailed representation of battery charging.

Problem instances

We model four instances (scenarios) of our world. Each instance increases the depth of the search space (more locations and more drones) while also restricting the possible solutions by introducing charging constraints and locking some locations until another task has been completed.

Instance 1

We have a model of our world with five locations, with the charging hub located at the center (charging). One drone has to inspect two sites and deliver a med kit to the last one. The drone starts empty and has to begin carrying the med kit since it is located at the same (depot) site the drone is, it also has a full charge. Every edge between locations has a distance cost of 1 and our goal is for the drone to deliver the med1 to site3, and inspect site1 and site2 .

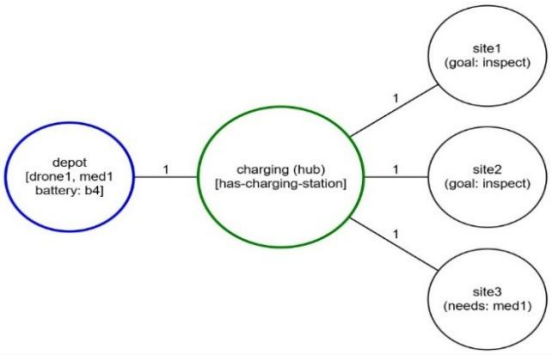


Figure 2: Diagram of Instance 1: Problem small.

Instance 2

The world now has two drones flying at the same time. There are eight locations in total, and the design of the model resembles a linear one. There are two charging stations (ch1, ch2) and five mission sites (site1, site2, site3, site4, and site5). One of them (site5) is locked until a drone inspects site4 first. Three of the mission sites require supplies (site1, site3, site5), while four of them require an inspection. The locations of the drones are as follows: drone1 is in the depot with the required supplies, and drone2 is in the charging hub (ch2). Both drones are fully charged (b6). Finally, the distance costs between locations vary between 1 and 2.

An **invalid action** thus would be any drone flying from site **ch2** to **site5** before inspecting **site4** since **(unlocked-by-inspection site4 site5).** This constraint is imposed because there's no predicate **(accessible site5)** in the **:init** part of the problem file and a precondition for the **fly** action is **(accessible ?to)**.

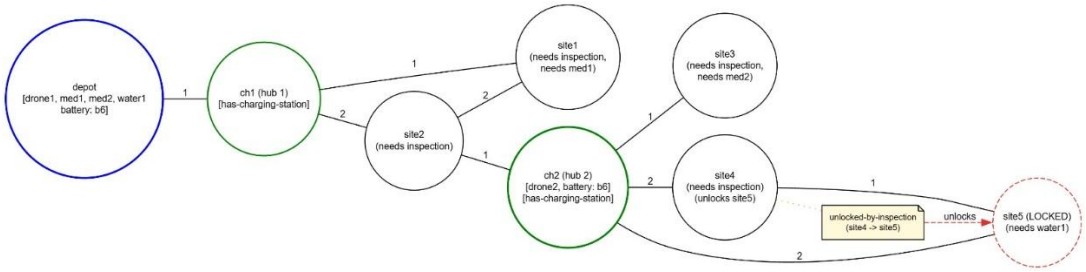


Figure 3: Diagram of Instance 2: Problem medium.

Instance 3

There are three drones flying at the same time and eleven locations in total. The design of the model is split into two distinct sectors (West Sector and East Sector) connected by a bridge. There are two charging stations (ch_west, ch_east) and seven mission sites (site_w1, site_w2, site_w3, site_e1, site_e2, site_e3, site_e4). One of them (site_e4) is locked until a drone delivers a supply to site_e2 first. Four of the mission sites require supplies (site_w2, site_w3, site_e2, site_e4), while the other three require an inspection (site_w1, site_e1, site_e3). The locations of the drones are as follows: drone1 is in the west depot (depot_west), alongside the required west supplies; drone2 is in the east charging hub (ch_east); and drone3 is in the east depot (depot_east), alongside the required east supplies. All three drones are fully charged (b6). Finally, the distance costs between locations vary from 1 to 3.

Instance 3 (1-Drone) shares the exact location distribution, flight costs, supply locations, and goal set as the standard 3-drone Large instance but has only a single drone.

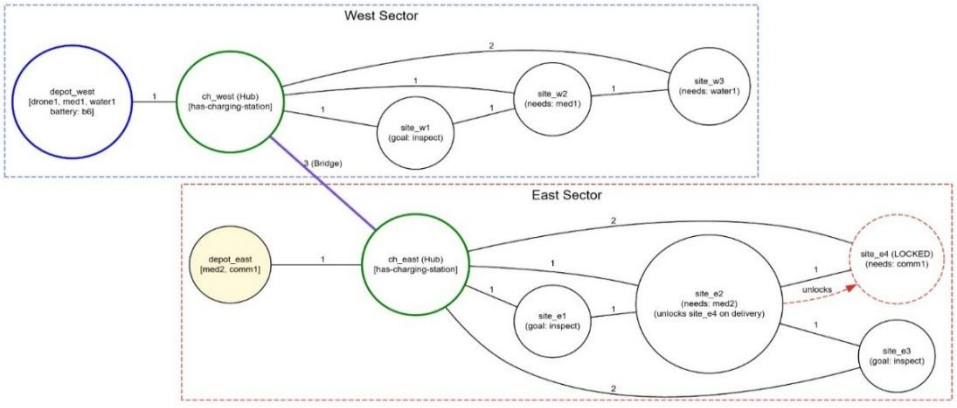


Figure 4. Diagram of Instance 3: Problem large – Three drones.

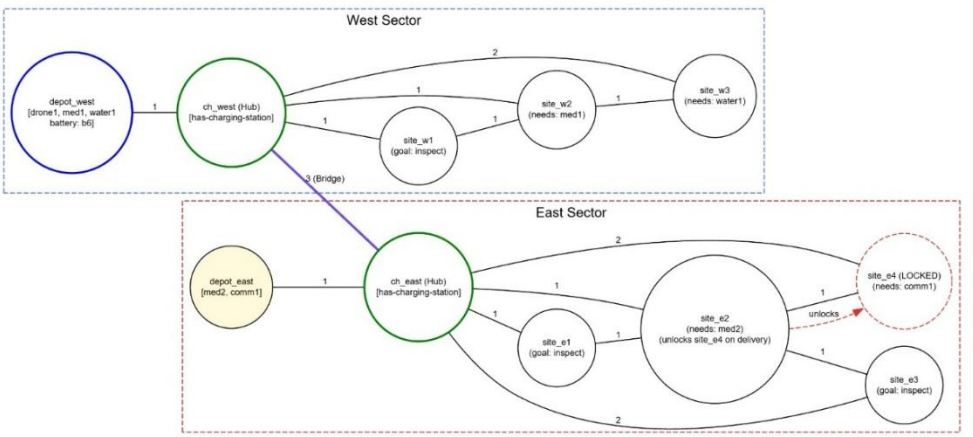


Figure 5. Diagram of Instance 3: Problem large – One drone.

Planner setup

We used the planning system, Fast Downward (Helmert, 2006) to generate plans for Domain A and Domain B in Ubuntu (WSL2). VAL (Howey, 2004) was used to validate the generated plans. Two planner configurations were tested: LAMA-first and A* with LM-cut. Both configurations were tested across the available problem instances for Domain A and Domain B. Each run had a time limit of 5 minutes and a memory limit of 4 GB.

For each experiment, the domain file, problem file and planner configuration were provided to Fast Downward. Generated plans were saved in my-plans/, and planner output was recorded in my-results/. The plans that were generated were then validated using VAL. Validation output was saved in my-validation/.

The experiments used the following command structure:

LAMA-first:

```
fast-downward.py --overall-time-limit 5m --overall-memory-limit 4G \
  --plan-file my-plans/domain-A-medium-lama-first.plan \
  --alias lama-first domain-A.pddl problem-medium.pddl
```

A* with LM-cut:

```
fast-downward.py --overall-time-limit 5m --overall-memory-limit 4G \
  --plan-file my-plans/domain-A-medium-astar-lmcut.plan \
  domain-A.pddl problem-medium.pddl --search "astar(lmcut())"
```

Plan validation:

```
Validate -v domain-A.pddl problem-medium.pddl \
  my-plans/domain-A-medium-lama-first.plan
```

Planner performance was evaluated using runtime, plan length, plan cost, expanded states and peak memory. VAL was used to check plan validity. Runs that reached the time limit without producing a plan were recorded as Solved = No.

Results and analysis

Instance	Domain	Planner	Solved	Runtime (s)	Search Time (s)	Plan Length	Plan Cost	Expanded States
Small	Domain-A	lama-first	Yes	0.22	0.00	11	11	30
Small	Domain-A	astar-lmcut	Yes	0.25	0.00	11	11	43
Medium	Domain-A	lama-first	Yes	0.25	0.00	35	42	79
Medium	Domain-A	astar-lmcut	Yes	13.05	12.79	28	32	184,237
Large	Domain-A	lama-first	Yes	0.26	0.01	32	35	77
Large	Domain-A	astar-lmcut	No	288.01	N/A	N/A	N/A	200,770
Large (1-Drone)	Domain-A	lama-first	Yes	0.29	0.01	44	56	100
Large (1-Drone)	Domain-A	astar-lmcut	Yes	0.42	0.18	35	39	4,614
Small	Domain-B	lama-first	Yes	0.24	0.00	12	12	31
Small	Domain-B	astar-lmcut	Yes	0.21	0.00	12	12	67
Medium	Domain-B	lama-first	Yes	0.33	0.10	48	55	10,480
Medium	Domain-B	astar-lmcut	Yes	21.83	21.58	34	38	446,517
Large	Domain-B	lama-first	Yes	0.29	0.01	52	61	485
Large	Domain-B	astar-lmcut	No	288.12	N/A	N/A	N/A	188,841
Large (1-Drone)	Domain-B	lama-first	Yes	0.24	0.01	51	55	2,162
Large (1-Drone)	Domain-B	astar-lmcut	Yes	0.56	0.29	48	53	7,528

Table 1. General results of every problem with both domains.

Problem-small analysis

Domain	Instance	Planner	Solved	Runtime (s)	Plan length	Plan cost	Expanded states
domain-A	small	lama-first	Yes	0.22	11	11	30
domain-A	small	astar-lmcut	Yes	0.25	11	11	43
domain-B	small	lama-first	Yes	0.24	12	12	31
domain-B	small	astar-lmcut	Yes	0.21	12	12	67

Table 2. Planning execution results for problem-small.

The problem-small results show that neither planner had any difficulty solving this instance. Every run finished almost instantly, averaging 0.23 seconds and requiring only 30 to 67 expanded states. The discrete battery encoding increases the state-space size, which explains why Domain B required more state expansions (up to 67 under A*) compared to Domain A (43 states). Despite this difference in the depth of search states, both domain models produced nearly identical runtimes and plan lengths and costs of 11 for Domain A and 12 for Domain B. This shows that, for small instances with space-search sizes of a few hundred (300 in this case), both planners perform correctly therefore cost-optimal ones like A* are preferred.

Due to the problem constraint, it's not possible for a plan to have no recharge actions, we can check this reading our A* LM-CUT plan below since it's the most optimal one (the shortest one) and see that there are 6 level 1 depletions in total, just by going to the first location and coming back, the drone battery depleted to b1, making it impossible to continue to other sites without recharging.

```
(pick-up-supply drone1 med1 depot)
(fly drone1 depot charging b4 b1 b3)
(fly drone1 charging site1 b3 b1 b2)
(inspect drone1 site1)
(fly drone1 site1 charging b2 b1 b1)
(recharge drone1 charging b1 b4)
(fly drone1 charging site2 b4 b1 b3)
(inspect drone1 site2)
(fly drone1 site2 charging b3 b1 b2)
(fly drone1 charging site3 b2 b1 b1)
(deliver-supply drone1 med1 site3)
; cost = 11 (unit cost)
```

Problem-medium analysis

Domain	Instance	Planner	Solved	Runtime (s)	Plan length	Plan cost	Expanded states
domain-A	medium	lama-first	Yes	0.25	35	42	79
domain-A	medium	astar-lmcut	Yes	13.05	28	32	184237
domain-B	medium	lama-first	Yes	0.33	48	55	10480
domain-B	medium	astar-lmcut	Yes	21.83	34	38	446517

Table 3. Planning execution results for problem-medium

On problem-medium, A* with LM-cut faced a substantial increase in search effort, expanding between 184,000 and 446,000 states and taking approximately 13–22 seconds. The discrete battery encoding in Domain B increased state expansions significantly, by more than 2.4 times for A* and more than 130 times for LAMA-first compared to Domain A. Comparing the two domain models shows that Domain A consistently achieved lower plan costs, with costs of 32 compared to 38 for A*. These results suggest that detailed battery increments introduce additional state-space states, which becomes more significant as the problem grows.

Problem-large analysis

Domain	Instance	Planner	Solved	Runtime (s)	Plan length	Plan cost	Expanded states
domain-A	large	lama-first	Yes	0.26	32	35	77
domain-A	large	astar-lmcut	No	288.01	N/A	N/A	200770
domain-B	large	lama-first	Yes	0.29	52	61	485
domain-B	large	astar-lmcut	No	288.12	N/A	N/A	188841

Table 4. Planning execution results for problem-large

On problem-large, A* with LM-cut timed out in both domains after approximately 288 seconds, having expanded between 188,000 and 200,000 states without finding a solution. This is because of the nature of the A* search, where it must compute the heuristics for every node in the search space.

The combination of a larger map and battery constraints created a search space that optimal search could not solve within the time limit. In contrast, LAMA-first found plans in both domains in under 0.3 seconds. However, Domain B produced longer plans (52 compared to 32) and higher plan costs (61 compared to 35) than Domain A. It also required approximately six times more state expansions. These results show that the choice of battery representation affects both search effort and the quality of the plans generated by LAMA-first.

Problem-large with only 1 Drone analysis

Domain	Instance	Planner	Solved	Runtime (s)	Plan length	Plan cost	Expanded states
domain-A	large-1drone	lama-first	Yes	0.29	44	56	100
domain-A	large-1drone	astar-lmcut	Yes	0.42	35	39	4614
domain-B	large-1drone	lama-first	Yes	0.24	51	55	2162
domain-B	large-1drone	astar-lmcut	Yes	0.56	48	53	7528

Table 4. Planning execution results for problem-large (1) Drone

Restricting the large instance to a single drone (problem-large-1drone) substantially improved A* with LM-cut performance. Both planners solved both domain models in under 0.6 seconds. Although the discrete battery encoding still required more state expansions (up to 7,528 in Domain B compared to 4,614 in Domain A under A*), the single-drone instance remained computationally manageable. Domain A also produced lower-cost plans (39 compared to 53) and shorter plans (35 compared to 48) than Domain B under A*. These results suggest that the number of drones explodes search performance, along with battery representation.

Problem-large 1 Drone vs 3 Drones analysis

Instance	Domain	Planner	Solved	Runtime (s)	Search Time (s)	Plan Length	Plan Cost	Expanded States
Large (3-Drones)	Domain-A	lama-first	Yes	0.26	0.01	32	35	77
Large (1-Drone)	Domain-A	lama-first	Yes	0.29	0.01	44	56	100

Table 5. Comparison of planning execution results for problem-large (1) Drone and problem-large (3) Drones.

We see that in the LAMA-first instance, the plan cost found was reduced from 56 to 35 when 3 drones were sent to complete the same tasks in problem-large, instead of just 1. The absolute plan cost reduction is 21, which translates to a 37.5% percentage decrease, as we see, this is not a substantial reduction taking into consideration that there are now 3 times more drones doing the same tasks, this is because classical planning is a sequential process and actions cannot be done in parallel. The reduction in cost lies in that it now takes less steps to reach a location since drones are more distributed.

Literature Connection

Mersheeva and Friedrich (2015) investigate mission planning for multiple UAVs with limited energy. They also include in their model the battery constraint for the drones, and although they do take into consideration that battery capacity can be heterogeneous among drones, their logic is

the same as ours when computing possible routes. They model a function called "NEEDBATCHANGE(v,Rv,p) that "checks if the vehicle v must change its battery before visiting point p*"* (Mersheeva et al., 2015 p. 351). This is the same logic in our preconditions for the fly action: (battery-after ?before ?c ?left).

For our problem, we calculate the most optimal path taking into consideration the cost and battery calculations between every trajectory (interconnection of two locations). Bit-Monnot et al. (2018) calculate this computation taking in consideration the same two factors, they state: "The energy consumption of a trajectory is defined as the sum of the consumption of all maneuvers in the trajectory and the energy needed to link any two subsequent maneuvers in the trajectory." (Bit-Monnot et al., 2018). In our case, we consider the cost of every action or step the drone does (which we stated was 1) and the battery depletion that happened between the flights, which is the fixed set of possible battery depletion combinations described in each problem file, i.e., (battery-after b4 b1 b3).

Limitations and future work

One limitation of our model is that Fast Downward does not support the numeric operations needed to directly update battery levels. Instead, we used discrete battery-level predicates, which made the model longer and harder to understand. It also required additional initial-state predicates in each problem file.

Other planning systems, such as Metric-FF (Hoffmann, 2003), support numeric state variables. Future work could solve the same problem using such a system and compare the model's simplicity and planner performance.

Another limitation is that all drones use the same maximum battery capacity. In real life, each drone may have different battery capacities, which would add more possible states to the planning problem. Future work could also include different battery capacities to make the model more realistic.

Appendix

Domain A's PDDL code

```
(define (domain emergency-drone)
  (:requirements :strips :typing :negative-preconditions :conditional-effects
:action-costs)

  (:types
    drone location supply level - object
    medkit water comms - supply
  )

  (:predicates
;LOCATION RELATED
    (at ?d - drone ?loc  - location) ;the current location of each drone
    (inspected ?loc - location) ;which connections have been inspected
     (unlocked-by-inspection ?source ?target - location)  ;inspecting a site
unlocks another one
    (unlocked-by-delivery ?source ?target - location) ;delivering a supply to
a source site unlocks a target one
     (connected ?from ?to - location) ;which locations are connected to each
other
    (accessible ?loc - location) ;true is the location currently unlocked
;LOCATION RELATED

;DRONE RELATED
    (empty ?d - drone)
    (carrying ?d - drone ?s - supply)
    (battery ?d - drone ?l - level)
;DRONE RELATED

;BATTERY RELATED
    (flight-cost ?from ?to - location ?c - level) ;
    (battery-after ?before ?c ?left - level)
    (next-level ?l1 ?l2 - level)
(max-level ?l - level)
;BATTERY RELATED

;CHARGING RELATED
     (has-charging-station ?loc - location) ;location where there is charging
station
;CHARGING RELATED


;SUPPLY RELATED
    (supply-at ?s - supply ?loc - location) ;which supply is currently in which
location
     (needs ?loc - location ?s - supply) ;serves as a requirement for some
location, supply ?s needs to be dropped at ?l
     (delivered ?s - supply ?loc - location) ;serves as a goal, the required
supply was delivered to the location it required it
;SUPPLY RELATED
  )

  (:functions
    (total-cost) - number
```

```
    (distance ?from ?to - location) - number
  )

;Moves a drone between connected nodes
; It first receives the name of the drone (?d), the current location (?from),
;the destination site (?to) and the state of the battery level:
;the current level (?before), what will the cost of the flight be (?c) and
;the battery that will be left (?left).
  (:action fly
    :parameters (?d - drone ?from ?to - location ?before ?c ?left - level)
    :precondition (and
      (at ?d ?from)
      (connected ?from ?to)
      (accessible ?to)
      (battery ?d ?before)
      (flight-cost ?from ?to ?c)
      (battery-after ?before ?c ?left)
    )
    :effect (and
      (not (at ?d ?from))
      (at ?d ?to)
      (not (battery ?d ?before))
      (battery ?d ?left)
      (increase (total-cost) (distance ?from ?to))
    )
  )

;Inspects a location
;It evaluates that the drone is at an uninspected location and
;after checking that needs inspection it does so; if the inspected
;site unblocks other locations, those targets are marked as accessible.
  (:action inspect
    :parameters (?d - drone ?loc - location)
    :precondition (and
      (at ?d ?loc)
      (not (inspected ?loc))
    )
    :effect (and
      (inspected ?loc)
      (forall (?y - location)
        (when (unlocked-by-inspection ?loc ?y)
          (accessible ?y)
        )
      )
      (increase (total-cost) 1)
    )
  )

;Pick up a supply package
;Allows a drone to collect a supply package given the drone
;is empty and at the same site as the supply. It updates the drone's status to
carrying.
  (:action pick-up-supply
    :parameters (?d - drone ?s - supply ?loc - location)
    :precondition (and
      (at ?d ?loc)
      (supply-at ?s ?loc)
```

```
      (empty ?d)
    )
    :effect (and
      (not (supply-at ?s ?loc))
      (not (empty ?d))
      (carrying ?d ?s)
      (increase (total-cost) 1)
    )
  )

;Deliver a supply package to a location that requires it
;It drops the required supply on the site that needed it
;given that the drone is carrying the specific supply required by the location.
  (:action deliver-supply
    :parameters (?d - drone ?s - supply ?loc - location)
    :precondition (and
      (at ?d ?loc)
      (carrying ?d ?s)
      (needs ?loc ?s)
    )
    :effect (and
      (not (carrying ?d ?s))
      (empty ?d)
      (delivered ?s ?loc)
      (supply-at ?s ?loc)
      (not (needs ?loc ?s))
      (forall (?y - location)
        (when (unlocked-by-delivery ?loc ?y)
          (accessible ?y)
        )
      )
      (increase (total-cost) 1)
    )
  )

;Recharge the battery to the maximum
;Restores a drone's battery when the drone is located at a node
;that has a charging station. In the Model A, the action instantly
;updates the drone's battery to the defined max-level in a single step,
;it does this removing the current battery (battery ?d ?cur) from the true facts
;and updates it with the max value listed in the problem file.
  (:action recharge
    :parameters (?d - drone ?loc - location ?cur ?max - level)
    :precondition (and
      (at ?d ?loc)
      (has-charging-station ?loc)
      (battery ?d ?cur)
      (max-level ?max)
    )
    :effect (and
      (not (battery ?d ?cur))
      (battery ?d ?max)
      (increase (total-cost) 1)
    )
  )
)
```

Domain B's PDDL code

```
(define (domain emergency-drone)
  (:requirements :strips :typing :negative-preconditions :conditional-effects
:action-costs)

  (:types
    drone location supply level - object
    medkit water comms - supply
  )

  (:predicates
;LOCATION RELATED
    (at ?d - drone ?loc  - location) ;the current location of each drone
    (inspected ?loc - location) ;which connections have been inspected
     (unlocked-by-inspection ?source ?target - location)  ;inspecting a site
unlocks another one
    (unlocked-by-delivery ?source ?target - location) ;delivering a supply to
a source site unlocks a target one
     (connected ?from ?to - location) ;which locations are connected to each
other
    (accessible ?loc - location) ;true is the location currently unlocked
;LOCATION RELATED

;DRONE RELATED
    (empty ?d - drone)  ;this is true if the drone isn't carrying anything
     (carrying ?d - drone ?s - supply)  ;describes what supply the drone is
carrying
    (battery ?d - drone ?l - level)  ; how much battery the drone has
;DRONE RELATED

;BATTERY RELATED
    (flight-cost ?from ?to - location ?c - level) ;looks for the cost of the
distance between two sites in the problem file
     (battery-after ?before ?c ?left - level)   ;looks up the next depletion
level of the battery according to the cost of the flight
    (next-level ?l1 ?l2 - level) ;looks up the next level of the battery while
charging in the problem file
    (max-level ?l - level)    ; looks up what the maximum battery level b is
for the drone
;BATTERY RELATED

;CHARGING RELATED
    (has-charging-station ?loc - location) ;location where there is charging
station
;CHARGING RELATED


;SUPPLY RELATED
   (supply-at ?s - supply ?loc - location) ;which supply is currently in which
location
     (needs ?loc - location ?s - supply) ;serves as a requirement for some
location, supply ?s needs to be dropped at ?l
     (delivered ?s - supply ?loc - location) ;serves as a goal, the required
supply was delivered to the location it required it
;SUPPLY RELATED
  )
```

```
  (:functions
    (total-cost) - number
    (distance ?from ?to - location) - number
  )

;Moves a drone between connected nodes
; It first receives the name of the drone (?d), the current location (?from),
;the destination site (?to) and the state of the battery level:
;the current level (?before), what will the cost of the flight be (?c) and
;the battery that will be left (?left).
  (:action fly
    :parameters (?d - drone ?from ?to - location ?before ?c ?left - level)
    :precondition (and
      (at ?d ?from)
      (connected ?from ?to)
      (accessible ?to)
      (battery ?d ?before)
      (flight-cost ?from ?to ?c)
      (battery-after ?before ?c ?left)
    )
    :effect (and
      (not (at ?d ?from))
      (at ?d ?to)
      (not (battery ?d ?before))
      (battery ?d ?left)
      (increase (total-cost) (distance ?from ?to))
    )
  )

;Inspects a location
;It evaluates that the drone is at an uninspected location and
;after checking that needs inspection it does so; if the inspected
;site unblocks other locations, those targets are marked as accessible.
  (:action inspect
    :parameters (?d - drone ?loc - location)
    :precondition (and
      (at ?d ?loc)
      (not (inspected ?loc))
    )
    :effect (and
      (inspected ?loc)
      (forall (?y - location)
        (when (unlocked-by-inspection ?loc ?y)
          (accessible ?y)
        )
      )
      (increase (total-cost) 1)
    )
  )

;Pick up a supply package
;Allows a drone to collect a supply package given the drone
;is empty and at the same site as the supply. It updates the drone's status to
carrying.
  (:action pick-up-supply
    :parameters (?d - drone ?s - supply ?loc - location)
```

```
    :precondition (and
      (at ?d ?loc)
      (supply-at ?s ?loc)
      (empty ?d)
    )
    :effect (and
      (not (supply-at ?s ?loc))
      (not (empty ?d))
      (carrying ?d ?s)
      (increase (total-cost) 1)
    )
  )

;Deliver a supply package to a location that requires it
;It drops the required supply on the site that needed it
;given that the drone is carrying the specific supply required by the location.
  (:action deliver-supply
    :parameters (?d - drone ?s - supply ?loc - location)
    :precondition (and
      (at ?d ?loc)
      (carrying ?d ?s)
      (needs ?loc ?s)
    )
    :effect (and
      (not (carrying ?d ?s))
      (empty ?d)
      (delivered ?s ?loc)
      (supply-at ?s ?loc)
      (not (needs ?loc ?s))
      (forall (?y - location)
        (when (unlocked-by-delivery ?loc ?y)
          (accessible ?y)
        )
      )
      (increase (total-cost) 1)
    )
  )

;Recharge the battery by one battery level per recharge
;it removes the current battery (battery ?d ?cur) from the true facts and
updates it
;with the ?next value listed in the problem file for the ?cur ?next value pair
(next-level ?cur ?next) predicate.
  (:action recharge
    :parameters (?d - drone ?loc - location ?cur ?next - level)
    :precondition (and
      (at ?d ?loc)
      (has-charging-station ?loc)
      (battery ?d ?cur)
      (next-level ?cur ?next)
    )
    :effect (and
      (not (battery ?d ?cur))
      (battery ?d ?next)
      (increase (total-cost) 1)
    )
  )
```

```
)
```

Problem-small PDDL

```
(define (problem emergency-small)
  (:domain emergency-drone)

  (:objects
    drone1 - drone
    depot charging site1 site2 site3 - location
    med1 - medkit
    b0 b1 b2 b3 b4 - level
  )

  (:init
;Initialize the cost of the plan in 0
    (= (total-cost) 0)

;Drone starts at depot with no supply attached and battery full at b4
    (at drone1 depot)
    (empty drone1)
    (battery drone1 b4)

;There is a charging station at charging
    (has-charging-station charging)

;ALL locations are unlocked from the start
    (accessible depot)
    (accessible charging)
    (accessible site1)
    (accessible site2)
    (accessible site3)

;Description of the sites interconnection
    (connected depot charging)
    (connected charging depot)
    (connected charging site1)
    (connected site1 charging)
    (connected charging site2)
    (connected site2 charging)
    (connected charging site3)
    (connected site3 charging)

;Distances cost 1
    (= (distance depot charging) 1)
    (= (distance charging depot) 1)
    (= (distance charging site1) 1)
    (= (distance site1 charging) 1)
    (= (distance charging site2) 1)
    (= (distance site2 charging) 1)
    (= (distance charging site3) 1)
    (= (distance site3 charging) 1)

;Flight battery costs (for b1)
    (flight-cost depot charging b1)
    (flight-cost charging depot b1)
    (flight-cost charging site1 b1)
```

```
    (flight-cost site1 charging b1)
    (flight-cost charging site2 b1)
    (flight-cost site2 charging b1)
    (flight-cost charging site3 b1)
    (flight-cost site3 charging b1)

;Supplies and mission requirements
    (supply-at med1 depot)
    (needs site3 med1)

;Discrete battery levels and transitions
    (max-level b4)
    (next-level b0 b1)
    (next-level b1 b2)
    (next-level b2 b3)
    (next-level b3 b4)

;Battery consumption transitions for flight-cost b1
    (battery-after b4 b1 b3)
    (battery-after b3 b1 b2)
    (battery-after b2 b1 b1)
    (battery-after b1 b1 b0)
  )

  (:goal (and
    (inspected site1)
    (inspected site2)
    (delivered med1 site3)
  ))

  (:metric minimize (total-cost))
)
```

Problem-medium PDDL file

```
(define (problem emergency-medium)
  (:domain emergency-drone)

  (:objects
    drone1 drone2 - drone
    depot ch1 ch2 site1 site2 site3 site4 site5 - location
    med1 med2 - medkit
    water1 - water
    b0 b1 b2 b3 b4 b5 b6 - level
  )

  (:init
;Initialize the cost of the plan in 0
    (= (total-cost) 0)

;Drones initial state
    (at drone1 depot)
    (empty drone1)
    (battery drone1 b6)

    (at drone2 ch2)
    (empty drone2)
    (battery drone2 b6)
```

```
;Charging infrastructure
    (has-charging-station ch1)
    (has-charging-station ch2)

;Accessibility: site5 is initially locked, all others open
    (accessible depot)
    (accessible ch1)
    (accessible ch2)
    (accessible site1)
    (accessible site2)
    (accessible site3)
    (accessible site4)
;NOT ACCESSIBLE - site5 is unlocked by inspecting neighbor site4
    (unlocked-by-inspection site4 site5)

;Description of the sites interconnection
    (connected depot ch1)
    (connected ch1 depot)

    (connected ch1 site1)
    (connected site1 ch1)

    (connected ch1 site2)
    (connected site2 ch1)

    (connected site1 site2)
    (connected site2 site1)

    (connected site2 ch2)
    (connected ch2 site2)

    (connected ch2 site3)
    (connected site3 ch2)

    (connected ch2 site4)
    (connected site4 ch2)

    (connected site4 site5)
    (connected site5 site4)

    (connected ch2 site5)
    (connected site5 ch2)

;Action cost distances (values 1 and 2)
    (= (distance depot ch1) 1)
    (= (distance ch1 depot) 1)

    (= (distance ch1 site1) 1)
    (= (distance site1 ch1) 1)

    (= (distance ch1 site2) 2)
    (= (distance site2 ch1) 2)

    (= (distance site1 site2) 2)
    (= (distance site2 site1) 2)
```

```
    (= (distance site2 ch2) 1)
    (= (distance ch2 site2) 1)

    (= (distance ch2 site3) 1)
    (= (distance site3 ch2) 1)

    (= (distance ch2 site4) 2)
    (= (distance site4 ch2) 2)

    (= (distance site4 site5) 1)
    (= (distance site5 site4) 1)

    (= (distance ch2 site5) 2)
    (= (distance site5 ch2) 2)

;Flight battery costs (for b1 and b2)
    (flight-cost depot ch1 b1)
    (flight-cost ch1 depot b1)

    (flight-cost ch1 site1 b1)
    (flight-cost site1 ch1 b1)

    (flight-cost ch1 site2 b2)
    (flight-cost site2 ch1 b2)

    (flight-cost site1 site2 b2)
    (flight-cost site2 site1 b2)

    (flight-cost site2 ch2 b1)
    (flight-cost ch2 site2 b1)

    (flight-cost ch2 site3 b1)
    (flight-cost site3 ch2 b1)

    (flight-cost ch2 site4 b2)
    (flight-cost site4 ch2 b2)

    (flight-cost site4 site5 b1)
    (flight-cost site5 site4 b1)

    (flight-cost ch2 site5 b2)
    (flight-cost site5 ch2 b2)

;Supplies and destination demands
    (supply-at med1 depot)
    (supply-at med2 depot)
    (supply-at water1 depot)

    (needs site1 med1)
    (needs site3 med2)
    (needs site5 water1)

;Discrete battery levels and transitions
    (max-level b6)
    (next-level b0 b1)
    (next-level b1 b2)
    (next-level b2 b3)
```

```
    (next-level b3 b4)
    (next-level b4 b5)
    (next-level b5 b6)

;Battery transitions for cost b1
    (battery-after b6 b1 b5)
    (battery-after b5 b1 b4)
    (battery-after b4 b1 b3)
    (battery-after b3 b1 b2)
    (battery-after b2 b1 b1)
    (battery-after b1 b1 b0)

;Battery transitions for cost b2
    (battery-after b6 b2 b4)
    (battery-after b5 b2 b3)
    (battery-after b4 b2 b2)
    (battery-after b3 b2 b1)
    (battery-after b2 b2 b0)
  )

  (:goal (and
    (inspected site1)
    (inspected site2)
    (inspected site3)
    (inspected site4)
    (delivered med1 site1)
    (delivered med2 site3)
    (delivered water1 site5)
  ))

  (:metric minimize (total-cost))
)
```

Problem-large

```
(define (problem emergency-large)
  (:domain emergency-drone)

  (:objects
    drone1 drone2 drone3 - drone
    depot_west depot_east ch_west ch_east - location
    site_w1 site_w2 site_w3 - location
    site_e1 site_e2 site_e3 site_e4 - location
    med1 med2 - medkit
    water1 - water
    comm1 - comms
    b0 b1 b2 b3 b4 b5 b6 - level
  )
  (:init
;Initialize the cost of the plan in 0
    (= (total-cost) 0)

;Set drone1 at depot_west
    (at drone1 depot_west)
    (empty drone1)
    (battery drone1 b6)

;Set drone2 at charging station east
```

```
    (at drone2 ch_east)
    (empty drone2)
    (battery drone2 b6)

;Set drone3 at depot_east
    (at drone3 depot_east)
    (empty drone3)
    (battery drone3 b6)

;There are two charging locations
    (has-charging-station ch_west)
    (has-charging-station ch_east)

;BATTERY LEVELS
    (max-level b6)
    (next-level b0 b1)
    (next-level b1 b2)
    (next-level b2 b3)
    (next-level b3 b4)
    (next-level b4 b5)
    (next-level b5 b6)

;ALL locations except site_e4 is open
    (accessible depot_west)
    (accessible depot_east)
    (accessible ch_west)
    (accessible ch_east)
    (accessible site_w1)
    (accessible site_w2)
    (accessible site_w3)
    (accessible site_e1)
    (accessible site_e2)
    (accessible site_e3)

;NOT ACCESSIBLE - site_e4 is unlocked by inspecting site_e2
    (unlocked-by-delivery site_e2 site_e4)

;Description of the sites interconnection
    (connected depot_west ch_west)
    (connected ch_west depot_west)

    (connected ch_west site_w1)
    (connected site_w1 ch_west)

    (connected ch_west site_w2)
    (connected site_w2 ch_west)

    (connected site_w1 site_w2)
    (connected site_w2 site_w1)

    (connected site_w2 site_w3)
    (connected site_w3 site_w2)

    (connected ch_west site_w3)
    (connected site_w3 ch_west)

;Site mapping of bridges
```

```
    (connected ch_west ch_east)
    (connected ch_east ch_west)

;Site mapping of East Sector
    (connected depot_east ch_east)
    (connected ch_east depot_east)

    (connected ch_east site_e1)
    (connected site_e1 ch_east)

    (connected ch_east site_e2)
    (connected site_e2 ch_east)

    (connected site_e1 site_e2)
    (connected site_e2 site_e1)

    (connected site_e2 site_e3)
    (connected site_e3 site_e2)

    (connected ch_east site_e3)
    (connected site_e3 ch_east)

    (connected site_e2 site_e4)
    (connected site_e4 site_e2)

    (connected ch_east site_e4)
    (connected site_e4 ch_east)

; Distances cost of West Sector
    (= (distance depot_west ch_west) 1)
    (= (distance ch_west depot_west) 1)

    (= (distance ch_west site_w1) 1)
    (= (distance site_w1 ch_west) 1)

    (= (distance ch_west site_w2) 1)
    (= (distance site_w2 ch_west) 1)

    (= (distance site_w1 site_w2) 1)
    (= (distance site_w2 site_w1) 1)

    (= (distance site_w2 site_w3) 1)
    (= (distance site_w3 site_w2) 1)

    (= (distance ch_west site_w3) 2)
    (= (distance site_w3 ch_west) 2)

; Distances cost of bridge
    (= (distance ch_west ch_east) 3)
    (= (distance ch_east ch_west) 3)

; Distances cost of East Sector
    (= (distance depot_east ch_east) 1)
    (= (distance ch_east depot_east) 1)

    (= (distance ch_east site_e1) 1)
    (= (distance site_e1 ch_east) 1)
```

```
    (= (distance ch_east site_e2) 1)
    (= (distance site_e2 ch_east) 1)

    (= (distance site_e1 site_e2) 1)
    (= (distance site_e2 site_e1) 1)

    (= (distance site_e2 site_e3) 1)
    (= (distance site_e3 site_e2) 1)

    (= (distance ch_east site_e3) 2)
    (= (distance site_e3 ch_east) 2)

    (= (distance site_e2 site_e4) 1)
    (= (distance site_e4 site_e2) 1)

    (= (distance ch_east site_e4) 2)
    (= (distance site_e4 ch_east) 2)

;Flight battery costs (for b1, b2 and b3)
    (flight-cost depot_west ch_west b1)
    (flight-cost ch_west depot_west b1)

    (flight-cost ch_west site_w1 b1)
    (flight-cost site_w1 ch_west b1)

    (flight-cost ch_west site_w2 b1)
    (flight-cost site_w2 ch_west b1)

    (flight-cost site_w1 site_w2 b1)
    (flight-cost site_w2 site_w1 b1)

    (flight-cost site_w2 site_w3 b1)
    (flight-cost site_w3 site_w2 b1)

    (flight-cost ch_west site_w3 b2)
    (flight-cost site_w3 ch_west b2)


    (flight-cost ch_west ch_east b3)
    (flight-cost ch_east ch_west b3)


    (flight-cost depot_east ch_east b1)
    (flight-cost ch_east depot_east b1)

    (flight-cost ch_east site_e1 b1)
    (flight-cost site_e1 ch_east b1)

    (flight-cost ch_east site_e2 b1)
    (flight-cost site_e2 ch_east b1)

    (flight-cost site_e1 site_e2 b1)
    (flight-cost site_e2 site_e1 b1)

    (flight-cost site_e2 site_e3 b1)
    (flight-cost site_e3 site_e2 b1)
```

```
    (flight-cost ch_east site_e3 b2)
    (flight-cost site_e3 ch_east b2)

    (flight-cost site_e2 site_e4 b1)
    (flight-cost site_e4 site_e2 b1)

    (flight-cost ch_east site_e4 b2)
    (flight-cost site_e4 ch_east b2)

;Supplies locations
    (supply-at med1 depot_west)
    (supply-at water1 depot_west)
    (supply-at med2 depot_east)
    (supply-at comm1 depot_east)
;Supplies requirements
    (needs site_w2 med1)
    (needs site_w3 water1)
    (needs site_e2 med2)
    (needs site_e4 comm1)

;BATTERY DEPLETIONS b1
    (battery-after b6 b1 b5)
    (battery-after b5 b1 b4)
    (battery-after b4 b1 b3)
    (battery-after b3 b1 b2)
    (battery-after b2 b1 b1)
    (battery-after b1 b1 b0)

;BATTERY DEPLETIONS b2
    (battery-after b6 b2 b4)
    (battery-after b5 b2 b3)
    (battery-after b4 b2 b2)
    (battery-after b3 b2 b1)
    (battery-after b2 b2 b0)

;BATTERY DEPLETIONS b3
    (battery-after b6 b3 b3)
    (battery-after b5 b3 b2)
    (battery-after b4 b3 b1)
    (battery-after b3 b3 b0)
  )
  (:goal (and
    (inspected site_w1)
    (inspected site_e1)
    (inspected site_e3)
    (delivered med1 site_w2)
    (delivered water1 site_w3)
    (delivered med2 site_e2)
    (delivered comm1 site_e4)
  ))

  (:metric minimize (total-cost)
)
)
```

Problem-large with one (1) drone PDDL file

```
(define (problem emergency-large-1drone)
  (:domain emergency-drone)

  (:objects
    drone1 - drone
    depot_west depot_east ch_west ch_east - location
    site_w1 site_w2 site_w3 - location
    site_e1 site_e2 site_e3 site_e4 - location
    b0 b1 b2 b3 b4 b5 b6 - level
    med1 med2 - medkit
    water1 - water
    comm1 - comms
  )

  (:init
;Initialize the cost of the plan in 0
    (= (total-cost) 0)

;Set dron in depot_west, full battery and no supplies attached
    (at drone1 depot_west)
    (empty drone1)
    (battery drone1 b6)

;Charging hubs location
    (has-charging-station ch_west)
    (has-charging-station ch_east)

;UNLOCKED SITES
    (accessible depot_west)
    (accessible depot_east)
    (accessible ch_west)
    (accessible ch_east)
    (accessible site_w1)
    (accessible site_w2)
    (accessible site_w3)
    (accessible site_e1)
    (accessible site_e2)
    (accessible site_e3)

;LOCKED SITE site_e4 unlocked by site_e2
    (unlocked-by-delivery site_e2 site_e4)

;DESCRIPTION OF SITE INTERCONNECTION
    (connected depot_west ch_west)
    (connected ch_west depot_west)

    (connected ch_west site_w1)
    (connected site_w1 ch_west)

    (connected ch_west site_w2)
    (connected site_w2 ch_west)

    (connected site_w1 site_w2)
    (connected site_w2 site_w1)
```

```
    (connected site_w2 site_w3)
    (connected site_w3 site_w2)

    (connected ch_west site_w3)
    (connected site_w3 ch_west)


;DESCRIPTION OF SITE INTERCONNECTION ON BRIDGE
    (connected ch_west ch_east)
    (connected ch_east ch_west)


;DESCRIPTION OF SITE INTERCONNECTION
    (connected depot_east ch_east)
    (connected ch_east depot_east)

    (connected ch_east site_e1)
    (connected site_e1 ch_east)

    (connected ch_east site_e2)
    (connected site_e2 ch_east)

    (connected site_e1 site_e2)
    (connected site_e2 site_e1)

    (connected site_e2 site_e3)
    (connected site_e3 site_e2)

    (connected ch_east site_e3)
    (connected site_e3 ch_east)

    (connected site_e2 site_e4)
    (connected site_e4 site_e2)

    (connected ch_east site_e4)
    (connected site_e4 ch_east)

;DISTANCES COST (costs 1, 2 and 3)
    (= (distance depot_west ch_west) 1)
    (= (distance ch_west depot_west) 1)

    (= (distance ch_west site_w1) 1)
    (= (distance site_w1 ch_west) 1)

    (= (distance ch_west site_w2) 1)
    (= (distance site_w2 ch_west) 1)

    (= (distance site_w1 site_w2) 1)
    (= (distance site_w2 site_w1) 1)

    (= (distance site_w2 site_w3) 1)
    (= (distance site_w3 site_w2) 1)

    (= (distance ch_west site_w3) 2)
    (= (distance site_w3 ch_west) 2)
```

```
    (= (distance ch_west ch_east) 3)
    (= (distance ch_east ch_west) 3)


    (= (distance depot_east ch_east) 1)
    (= (distance ch_east depot_east) 1)

    (= (distance ch_east site_e1) 1)
    (= (distance site_e1 ch_east) 1)

    (= (distance ch_east site_e2) 1)
    (= (distance site_e2 ch_east) 1)

    (= (distance site_e1 site_e2) 1)
    (= (distance site_e2 site_e1) 1)

    (= (distance site_e2 site_e3) 1)
    (= (distance site_e3 site_e2) 1)

    (= (distance ch_east site_e3) 2)
    (= (distance site_e3 ch_east) 2)

    (= (distance site_e2 site_e4) 1)
    (= (distance site_e4 site_e2) 1)

    (= (distance ch_east site_e4) 2)
    (= (distance site_e4 ch_east) 2)

;FLIGHT BATTERY COST (for b1, b2 and b3)
    (flight-cost depot_west ch_west b1)
    (flight-cost ch_west depot_west b1)

    (flight-cost ch_west site_w1 b1)
    (flight-cost site_w1 ch_west b1)

    (flight-cost ch_west site_w2 b1)
    (flight-cost site_w2 ch_west b1)

    (flight-cost site_w1 site_w2 b1)
    (flight-cost site_w2 site_w1 b1)

    (flight-cost site_w2 site_w3 b1)
    (flight-cost site_w3 site_w2 b1)

    (flight-cost ch_west site_w3 b2)
    (flight-cost site_w3 ch_west b2)


    (flight-cost ch_west ch_east b3)
    (flight-cost ch_east ch_west b3)


    (flight-cost depot_east ch_east b1)
    (flight-cost ch_east depot_east b1)

    (flight-cost ch_east site_e1 b1)
    (flight-cost site_e1 ch_east b1)
```

```
    (flight-cost ch_east site_e2 b1)
    (flight-cost site_e2 ch_east b1)

    (flight-cost site_e1 site_e2 b1)
    (flight-cost site_e2 site_e1 b1)

    (flight-cost site_e2 site_e3 b1)
    (flight-cost site_e3 site_e2 b1)

    (flight-cost ch_east site_e3 b2)
    (flight-cost site_e3 ch_east b2)

    (flight-cost site_e2 site_e4 b1)
    (flight-cost site_e4 site_e2 b1)

    (flight-cost ch_east site_e4 b2)
    (flight-cost site_e4 ch_east b2)

;SUPPLY locations
    (supply-at med1 depot_west)
    (supply-at water1 depot_west)
    (supply-at med2 depot_east)
    (supply-at comm1 depot_east)
;Supply requirements
    (needs site_w2 med1)
    (needs site_w3 water1)
    (needs site_e2 med2)
    (needs site_e4 comm1)

;MAX Battery and recharging level mapping
    (max-level b6)
    (next-level b0 b1)
    (next-level b1 b2)
    (next-level b2 b3)
    (next-level b3 b4)
    (next-level b4 b5)
    (next-level b5 b6)

;Battery consumption transitions for b1
    (battery-after b6 b1 b5)
    (battery-after b5 b1 b4)
    (battery-after b4 b1 b3)
    (battery-after b3 b1 b2)
    (battery-after b2 b1 b1)
    (battery-after b1 b1 b0)

;Battery consumption transitions for b2
    (battery-after b6 b2 b4)
    (battery-after b5 b2 b3)
    (battery-after b4 b2 b2)
    (battery-after b3 b2 b1)
    (battery-after b2 b2 b0)

;Battery consumption transitions for b3
    (battery-after b6 b3 b3)
    (battery-after b5 b3 b2)
```

```
    (battery-after b4 b3 b1)
    (battery-after b3 b3 b0)
  )

  (:goal (and
    (inspected site_w1)
    (inspected site_e1)
    (inspected site_e3)
    (delivered med1 site_w2)
    (delivered water1 site_w3)
    (delivered med2 site_e2)
    (delivered comm1 site_e4)
  ))

  (:metric minimize (total-cost))
)
```

Plan created for the Instance 3 problem using Domain A and one (1) drone model

```
domain-A-large-1drone-astar-lmcut.plan
(pick-up-supply drone1 water1 depot_west)
(fly drone1 depot_west ch_west b6 b1 b5)
(fly drone1 ch_west site_w1 b5 b1 b4)
(inspect drone1 site_w1)
(fly drone1 site_w1 site_w2 b4 b1 b3)
(fly drone1 site_w2 site_w3 b3 b1 b2)
(deliver-supply drone1 water1 site_w3)
(fly drone1 site_w3 ch_west b2 b2 b0)
(recharge drone1 ch_west b0 b6)
(fly drone1 ch_west depot_west b6 b1 b5)
(pick-up-supply drone1 med1 depot_west)
(fly drone1 depot_west ch_west b5 b1 b4)
(fly drone1 ch_west site_w2 b4 b1 b3)
(deliver-supply drone1 med1 site_w2)
(fly drone1 site_w2 ch_west b3 b1 b2)
(recharge drone1 ch_west b2 b6)
(fly drone1 ch_west ch_east b6 b3 b3)
(fly drone1 ch_east depot_east b3 b1 b2)
(pick-up-supply drone1 med2 depot_east)
(fly drone1 depot_east ch_east b2 b1 b1)
(recharge drone1 ch_east b1 b6)
(fly drone1 ch_east site_e1 b6 b1 b5)
(inspect drone1 site_e1)
(fly drone1 site_e1 site_e2 b5 b1 b4)
(deliver-supply drone1 med2 site_e2)
(fly drone1 site_e2 ch_east b4 b1 b3)
(fly drone1 ch_east depot_east b3 b1 b2)
(pick-up-supply drone1 comm1 depot_east)
(fly drone1 depot_east ch_east b2 b1 b1)
(recharge drone1 ch_east b1 b6)
(fly drone1 ch_east site_e4 b6 b2 b4)
(deliver-supply drone1 comm1 site_e4)
(fly drone1 site_e4 site_e2 b4 b1 b3)
(fly drone1 site_e2 site_e3 b3 b1 b2)
(inspect drone1 site_e3)
; cost = 39 (general cost)
```

Val output for problem large with Domain A and A* planner (domain-A-large-1drone-astar-lmcut)

```
Type-checking fly
...action passes type checking.
Type-checking inspect
...action passes type checking.
Type-checking pick-up-supply
...action passes type checking.
Type-checking deliver-supply
...action passes type checking.
Type-checking recharge
...action passes type checking.
Checking plan: my-plans/domain-A-large-1drone-astar-lmcut.plan
Plan to validate:

Plan size: 35
1:
(pick-up-supply drone1 water1 depot_west)

2:
(fly drone1 depot_west ch_west b6 b1 b5)

3:
(fly drone1 ch_west site_w1 b5 b1 b4)

4:
(inspect drone1 site_w1)

5:
(fly drone1 site_w1 site_w2 b4 b1 b3)

6:
(fly drone1 site_w2 site_w3 b3 b1 b2)

7:
(deliver-supply drone1 water1 site_w3)

8:
(fly drone1 site_w3 ch_west b2 b2 b0)

9:
(recharge drone1 ch_west b0 b6)

10:
(fly drone1 ch_west depot_west b6 b1 b5)

11:
(pick-up-supply drone1 med1 depot_west)

12:
(fly drone1 depot_west ch_west b5 b1 b4)

13:
(fly drone1 ch_west site_w2 b4 b1 b3)
```

```
14:
(deliver-supply drone1 med1 site_w2)

15:
(fly drone1 site_w2 ch_west b3 b1 b2)

16:
(recharge drone1 ch_west b2 b6)

17:
(fly drone1 ch_west ch_east b6 b3 b3)

18:
(fly drone1 ch_east depot_east b3 b1 b2)

19:
(pick-up-supply drone1 med2 depot_east)

20:
(fly drone1 depot_east ch_east b2 b1 b1)

21:
(recharge drone1 ch_east b1 b6)

22:
(fly drone1 ch_east site_e1 b6 b1 b5)

23:
(inspect drone1 site_e1)

24:
(fly drone1 site_e1 site_e2 b5 b1 b4)

25:
(deliver-supply drone1 med2 site_e2)

26:
(fly drone1 site_e2 ch_east b4 b1 b3)

27:
(fly drone1 ch_east depot_east b3 b1 b2)

28:
(pick-up-supply drone1 comm1 depot_east)

29:
(fly drone1 depot_east ch_east b2 b1 b1)

30:
(recharge drone1 ch_east b1 b6)

31:
(fly drone1 ch_east site_e4 b6 b2 b4)

32:
(deliver-supply drone1 comm1 site_e4)
```

```
33:
(fly drone1 site_e4 site_e2 b4 b1 b3)

34:
(fly drone1 site_e2 site_e3 b3 b1 b2)

35:
(inspect drone1 site_e3)


Plan Validation details
-----------------------

Checking next happening (time 1)
Deleting (supply-at water1 depot_west)
Deleting (empty drone1)
Adding (carrying drone1 water1)
Updating (total-cost) (0) by 1 increase

Checking next happening (time 2)
Deleting (at drone1 depot_west)
Deleting (battery drone1 b6)
Adding (at drone1 ch_west)
Adding (battery drone1 b5)
Updating (total-cost) (1) by 1 increase

Checking next happening (time 3)
Deleting (at drone1 ch_west)
Deleting (battery drone1 b5)
Adding (at drone1 site_w1)
Adding (battery drone1 b4)
Updating (total-cost) (2) by 1 increase

Checking next happening (time 4)
Adding (inspected site_w1)
Updating (total-cost) (3) by 1 increase

Checking next happening (time 5)
Deleting (at drone1 site_w1)
Deleting (battery drone1 b4)
Adding (at drone1 site_w2)
Adding (battery drone1 b3)
Updating (total-cost) (4) by 1 increase

Checking next happening (time 6)
Deleting (at drone1 site_w2)
Deleting (battery drone1 b3)
Adding (at drone1 site_w3)
Adding (battery drone1 b2)
Updating (total-cost) (5) by 1 increase

Checking next happening (time 7)
Deleting (carrying drone1 water1)
Deleting (needs site_w3 water1)
Adding (empty drone1)
Adding (delivered water1 site_w3)
```

```
Adding (supply-at water1 site_w3)
Updating (total-cost) (6) by 1 increase

Checking next happening (time 8)
Deleting (at drone1 site_w3)
Deleting (battery drone1 b2)
Adding (at drone1 ch_west)
Adding (battery drone1 b0)
Updating (total-cost) (7) by 2 increase

Checking next happening (time 9)
Deleting (battery drone1 b0)
Adding (battery drone1 b6)
Updating (total-cost) (9) by 1 increase

Checking next happening (time 10)
Deleting (at drone1 ch_west)
Deleting (battery drone1 b6)
Adding (at drone1 depot_west)
Adding (battery drone1 b5)
Updating (total-cost) (10) by 1 increase

Checking next happening (time 11)
Deleting (supply-at med1 depot_west)
Deleting (empty drone1)
Adding (carrying drone1 med1)
Updating (total-cost) (11) by 1 increase

Checking next happening (time 12)
Deleting (at drone1 depot_west)
Deleting (battery drone1 b5)
Adding (at drone1 ch_west)
Adding (battery drone1 b4)
Updating (total-cost) (12) by 1 increase

Checking next happening (time 13)
Deleting (at drone1 ch_west)
Deleting (battery drone1 b4)
Adding (at drone1 site_w2)
Adding (battery drone1 b3)
Updating (total-cost) (13) by 1 increase

Checking next happening (time 14)
Deleting (carrying drone1 med1)
Deleting (needs site_w2 med1)
Adding (empty drone1)
Adding (delivered med1 site_w2)
Adding (supply-at med1 site_w2)
Updating (total-cost) (14) by 1 increase

Checking next happening (time 15)
Deleting (at drone1 site_w2)
Deleting (battery drone1 b3)
Adding (at drone1 ch_west)
Adding (battery drone1 b2)
Updating (total-cost) (15) by 1 increase
```

```
Checking next happening (time 16)
Deleting (battery drone1 b2)
Adding (battery drone1 b6)
Updating (total-cost) (16) by 1 increase

Checking next happening (time 17)
Deleting (at drone1 ch_west)
Deleting (battery drone1 b6)
Adding (at drone1 ch_east)
Adding (battery drone1 b3)
Updating (total-cost) (17) by 3 increase

Checking next happening (time 18)
Deleting (at drone1 ch_east)
Deleting (battery drone1 b3)
Adding (at drone1 depot_east)
Adding (battery drone1 b2)
Updating (total-cost) (20) by 1 increase

Checking next happening (time 19)
Deleting (supply-at med2 depot_east)
Deleting (empty drone1)
Adding (carrying drone1 med2)
Updating (total-cost) (21) by 1 increase

Checking next happening (time 20)
Deleting (at drone1 depot_east)
Deleting (battery drone1 b2)
Adding (at drone1 ch_east)
Adding (battery drone1 b1)
Updating (total-cost) (22) by 1 increase

Checking next happening (time 21)
Deleting (battery drone1 b1)
Adding (battery drone1 b6)
Updating (total-cost) (23) by 1 increase

Checking next happening (time 22)
Deleting (at drone1 ch_east)
Deleting (battery drone1 b6)
Adding (at drone1 site_e1)
Adding (battery drone1 b5)
Updating (total-cost) (24) by 1 increase

Checking next happening (time 23)
Adding (inspected site_e1)
Updating (total-cost) (25) by 1 increase

Checking next happening (time 24)
Deleting (at drone1 site_e1)
Deleting (battery drone1 b5)
Adding (at drone1 site_e2)
Adding (battery drone1 b4)
Updating (total-cost) (26) by 1 increase

Checking next happening (time 25)
Deleting (carrying drone1 med2)
```

```
Deleting (needs site_e2 med2)
Adding (empty drone1)
Adding (delivered med2 site_e2)
Adding (supply-at med2 site_e2)
Adding (accessible site_e4)
Updating (total-cost) (27) by 1 increase

Checking next happening (time 26)
Deleting (at drone1 site_e2)
Deleting (battery drone1 b4)
Adding (at drone1 ch_east)
Adding (battery drone1 b3)
Updating (total-cost) (28) by 1 increase

Checking next happening (time 27)
Deleting (at drone1 ch_east)
Deleting (battery drone1 b3)
Adding (at drone1 depot_east)
Adding (battery drone1 b2)
Updating (total-cost) (29) by 1 increase

Checking next happening (time 28)
Deleting (supply-at comm1 depot_east)
Deleting (empty drone1)
Adding (carrying drone1 comm1)
Updating (total-cost) (30) by 1 increase

Checking next happening (time 29)
Deleting (at drone1 depot_east)
Deleting (battery drone1 b2)
Adding (at drone1 ch_east)
Adding (battery drone1 b1)
Updating (total-cost) (31) by 1 increase

Checking next happening (time 30)
Deleting (battery drone1 b1)
Adding (battery drone1 b6)
Updating (total-cost) (32) by 1 increase

Checking next happening (time 31)
Deleting (at drone1 ch_east)
Deleting (battery drone1 b6)
Adding (at drone1 site_e4)
Adding (battery drone1 b4)
Updating (total-cost) (33) by 2 increase

Checking next happening (time 32)
Deleting (carrying drone1 comm1)
Deleting (needs site_e4 comm1)
Adding (empty drone1)
Adding (delivered comm1 site_e4)
Adding (supply-at comm1 site_e4)
Updating (total-cost) (35) by 1 increase

Checking next happening (time 33)
Deleting (at drone1 site_e4)
Deleting (battery drone1 b4)
```

```
Adding (at drone1 site_e2)
Adding (battery drone1 b3)
Updating (total-cost) (36) by 1 increase

Checking next happening (time 34)
Deleting (at drone1 site_e2)
Deleting (battery drone1 b3)
Adding (at drone1 site_e3)
Adding (battery drone1 b2)
Updating (total-cost) (37) by 1 increase

Checking next happening (time 35)
Adding (inspected site_e3)
Updating (total-cost) (38) by 1 increase
Plan executed successfully - checking goal
Plan valid
Final value: 39

Successful plans:
Value: 39
 my-plans/domain-A-large-1drone-astar-lmcut.plan 39
```

Val output for problem small with Domain B and A* planner- domain-B-small-astar-lmcut-val

```
Type-checking fly
...action passes type checking.
Type-checking inspect
...action passes type checking.
Type-checking pick-up-supply
...action passes type checking.
Type-checking deliver-supply
...action passes type checking.
Type-checking recharge
...action passes type checking.
Checking plan: my-plans/domain-B-small-astar-lmcut.plan
Plan to validate:

Plan size: 12
1:
(pick-up-supply drone1 med1 depot)

2:
(fly drone1 depot charging b4 b1 b3)

3:
(fly drone1 charging site1 b3 b1 b2)

4:
(inspect drone1 site1)

5:
(fly drone1 site1 charging b2 b1 b1)
```

```
6:
(recharge drone1 charging b1 b2)

7:
(fly drone1 charging site2 b2 b1 b1)

8:
(inspect drone1 site2)

9:
(fly drone1 site2 charging b1 b1 b0)

10:
(recharge drone1 charging b0 b1)

11:
(fly drone1 charging site3 b1 b1 b0)

12:
(deliver-supply drone1 med1 site3)


Plan Validation details
-----------------------

Checking next happening (time 1)
Deleting (supply-at med1 depot)
Deleting (empty drone1)
Adding (carrying drone1 med1)
Updating (total-cost) (0) by 1 increase

Checking next happening (time 2)
Deleting (at drone1 depot)
Deleting (battery drone1 b4)
Adding (at drone1 charging)
Adding (battery drone1 b3)
Updating (total-cost) (1) by 1 increase

Checking next happening (time 3)
Deleting (at drone1 charging)
Deleting (battery drone1 b3)
Adding (at drone1 site1)
Adding (battery drone1 b2)
Updating (total-cost) (2) by 1 increase

Checking next happening (time 4)
Adding (inspected site1)
Updating (total-cost) (3) by 1 increase

Checking next happening (time 5)
Deleting (at drone1 site1)
Deleting (battery drone1 b2)
Adding (at drone1 charging)
Adding (battery drone1 b1)
Updating (total-cost) (4) by 1 increase

Checking next happening (time 6)
```

```
Deleting (battery drone1 b1)
Adding (battery drone1 b2)
Updating (total-cost) (5) by 1 increase

Checking next happening (time 7)
Deleting (at drone1 charging)
Deleting (battery drone1 b2)
Adding (at drone1 site2)
Adding (battery drone1 b1)
Updating (total-cost) (6) by 1 increase

Checking next happening (time 8)
Adding (inspected site2)
Updating (total-cost) (7) by 1 increase

Checking next happening (time 9)
Deleting (at drone1 site2)
Deleting (battery drone1 b1)
Adding (at drone1 charging)
Adding (battery drone1 b0)
Updating (total-cost) (8) by 1 increase

Checking next happening (time 10)
Deleting (battery drone1 b0)
Adding (battery drone1 b1)
Updating (total-cost) (9) by 1 increase

Checking next happening (time 11)
Deleting (at drone1 charging)
Deleting (battery drone1 b1)
Adding (at drone1 site3)
Adding (battery drone1 b0)
Updating (total-cost) (10) by 1 increase

Checking next happening (time 12)
Deleting (carrying drone1 med1)
Deleting (needs site3 med1)
Adding (empty drone1)
Adding (delivered med1 site3)
Adding (supply-at med1 site3)
Updating (total-cost) (11) by 1 increase
Plan executed successfully - checking goal
Plan valid
Final value: 12

Successful plans:
Value: 12
 my-plans/domain-B-small-astar-lmcut.plan 12
```

Val output of Problem medium with Domain A and LAMA first planner - domain-A-medium-lama-first-val

```
Type-checking fly
...action passes type checking.
Type-checking inspect
...action passes type checking.
```

```
Type-checking pick-up-supply
...action passes type checking.
Type-checking deliver-supply
...action passes type checking.
Type-checking recharge
...action passes type checking.
Checking plan: plans/domain-A-medium-lama-first.plan
Plan to validate:

Plan size: 35
1:
(pick-up-supply drone1 med1 depot)

2:
(fly drone1 depot ch1 b6 b1 b5)

3:
(fly drone1 ch1 site1 b5 b1 b4)

4:
(inspect drone1 site1)

5:
(deliver-supply drone1 med1 site1)

6:
(fly drone1 site1 site2 b4 b2 b2)

7:
(inspect drone1 site2)

8:
(fly drone1 site2 ch1 b2 b2 b0)

9:
(fly drone2 ch2 site3 b6 b1 b5)

10:
(inspect drone2 site3)

11:
(fly drone2 site3 ch2 b5 b1 b4)

12:
(fly drone2 ch2 site4 b4 b2 b2)

13:
(inspect drone2 site4)

14:
(fly drone2 site4 site5 b2 b1 b1)

15:
(recharge drone1 ch1 b0 b6)

16:
(fly drone1 ch1 depot b6 b1 b5)
```

```
17:
(pick-up-supply drone1 med2 depot)

18:
(fly drone1 depot ch1 b5 b1 b4)

19:
(fly drone1 ch1 site2 b4 b2 b2)

20:
(fly drone1 site2 ch2 b2 b1 b1)

21:
(recharge drone1 ch2 b1 b6)

22:
(fly drone1 ch2 site3 b6 b1 b5)

23:
(deliver-supply drone1 med2 site3)

24:
(fly drone1 site3 ch2 b5 b1 b4)

25:
(fly drone1 ch2 site2 b4 b1 b3)

26:
(fly drone1 site2 ch1 b3 b2 b1)

27:
(recharge drone1 ch1 b1 b6)

28:
(fly drone1 ch1 depot b6 b1 b5)

29:
(pick-up-supply drone1 water1 depot)

30:
(fly drone1 depot ch1 b5 b1 b4)

31:
(fly drone1 ch1 site2 b4 b2 b2)

32:
(fly drone1 site2 ch2 b2 b1 b1)

33:
(recharge drone1 ch2 b1 b6)

34:
(fly drone1 ch2 site5 b6 b2 b4)

35:
(deliver-supply drone1 water1 site5)
```

```
Plan Validation details
-----------------------

Checking next happening (time 1)
Deleting (supply-at med1 depot)
Deleting (empty drone1)
Adding (carrying drone1 med1)
Updating (total-cost) (0) by 1 increase

Checking next happening (time 2)
Deleting (at drone1 depot)
Deleting (battery drone1 b6)
Adding (at drone1 ch1)
Adding (battery drone1 b5)
Updating (total-cost) (1) by 1 increase

Checking next happening (time 3)
Deleting (at drone1 ch1)
Deleting (battery drone1 b5)
Adding (at drone1 site1)
Adding (battery drone1 b4)
Updating (total-cost) (2) by 1 increase

Checking next happening (time 4)
Adding (inspected site1)
Updating (total-cost) (3) by 1 increase

Checking next happening (time 5)
Deleting (carrying drone1 med1)
Deleting (needs site1 med1)
Adding (empty drone1)
Adding (delivered med1 site1)
Adding (supply-at med1 site1)
Updating (total-cost) (4) by 1 increase

Checking next happening (time 6)
Deleting (at drone1 site1)
Deleting (battery drone1 b4)
Adding (at drone1 site2)
Adding (battery drone1 b2)
Updating (total-cost) (5) by 2 increase

Checking next happening (time 7)
Adding (inspected site2)
Updating (total-cost) (7) by 1 increase

Checking next happening (time 8)
Deleting (at drone1 site2)
Deleting (battery drone1 b2)
Adding (at drone1 ch1)
Adding (battery drone1 b0)
Updating (total-cost) (8) by 2 increase

Checking next happening (time 9)
Deleting (at drone2 ch2)
```

```
Deleting (battery drone2 b6)
Adding (at drone2 site3)
Adding (battery drone2 b5)
Updating (total-cost) (10) by 1 increase

Checking next happening (time 10)
Adding (inspected site3)
Updating (total-cost) (11) by 1 increase

Checking next happening (time 11)
Deleting (at drone2 site3)
Deleting (battery drone2 b5)
Adding (at drone2 ch2)
Adding (battery drone2 b4)
Updating (total-cost) (12) by 1 increase

Checking next happening (time 12)
Deleting (at drone2 ch2)
Deleting (battery drone2 b4)
Adding (at drone2 site4)
Adding (battery drone2 b2)
Updating (total-cost) (13) by 2 increase

Checking next happening (time 13)
Adding (inspected site4)
Adding (accessible site5)
Updating (total-cost) (15) by 1 increase

Checking next happening (time 14)
Deleting (at drone2 site4)
Deleting (battery drone2 b2)
Adding (at drone2 site5)
Adding (battery drone2 b1)
Updating (total-cost) (16) by 1 increase

Checking next happening (time 15)
Deleting (battery drone1 b0)
Adding (battery drone1 b6)
Updating (total-cost) (17) by 1 increase

Checking next happening (time 16)
Deleting (at drone1 ch1)
Deleting (battery drone1 b6)
Adding (at drone1 depot)
Adding (battery drone1 b5)
Updating (total-cost) (18) by 1 increase

Checking next happening (time 17)
Deleting (supply-at med2 depot)
Deleting (empty drone1)
Adding (carrying drone1 med2)
Updating (total-cost) (19) by 1 increase

Checking next happening (time 18)
Deleting (at drone1 depot)
Deleting (battery drone1 b5)
Adding (at drone1 ch1)
```

```
Adding (battery drone1 b4)
Updating (total-cost) (20) by 1 increase

Checking next happening (time 19)
Deleting (at drone1 ch1)
Deleting (battery drone1 b4)
Adding (at drone1 site2)
Adding (battery drone1 b2)
Updating (total-cost) (21) by 2 increase

Checking next happening (time 20)
Deleting (at drone1 site2)
Deleting (battery drone1 b2)
Adding (at drone1 ch2)
Adding (battery drone1 b1)
Updating (total-cost) (23) by 1 increase

Checking next happening (time 21)
Deleting (battery drone1 b1)
Adding (battery drone1 b6)
Updating (total-cost) (24) by 1 increase

Checking next happening (time 22)
Deleting (at drone1 ch2)
Deleting (battery drone1 b6)
Adding (at drone1 site3)
Adding (battery drone1 b5)
Updating (total-cost) (25) by 1 increase

Checking next happening (time 23)
Deleting (carrying drone1 med2)
Deleting (needs site3 med2)
Adding (empty drone1)
Adding (delivered med2 site3)
Adding (supply-at med2 site3)
Updating (total-cost) (26) by 1 increase

Checking next happening (time 24)
Deleting (at drone1 site3)
Deleting (battery drone1 b5)
Adding (at drone1 ch2)
Adding (battery drone1 b4)
Updating (total-cost) (27) by 1 increase

Checking next happening (time 25)
Deleting (at drone1 ch2)
Deleting (battery drone1 b4)
Adding (at drone1 site2)
Adding (battery drone1 b3)
Updating (total-cost) (28) by 1 increase

Checking next happening (time 26)
Deleting (at drone1 site2)
Deleting (battery drone1 b3)
Adding (at drone1 ch1)
Adding (battery drone1 b1)
Updating (total-cost) (29) by 2 increase
```

```
Checking next happening (time 27)
Deleting (battery drone1 b1)
Adding (battery drone1 b6)
Updating (total-cost) (31) by 1 increase

Checking next happening (time 28)
Deleting (at drone1 ch1)
Deleting (battery drone1 b6)
Adding (at drone1 depot)
Adding (battery drone1 b5)
Updating (total-cost) (32) by 1 increase

Checking next happening (time 29)
Deleting (supply-at water1 depot)
Deleting (empty drone1)
Adding (carrying drone1 water1)
Updating (total-cost) (33) by 1 increase

Checking next happening (time 30)
Deleting (at drone1 depot)
Deleting (battery drone1 b5)
Adding (at drone1 ch1)
Adding (battery drone1 b4)
Updating (total-cost) (34) by 1 increase

Checking next happening (time 31)
Deleting (at drone1 ch1)
Deleting (battery drone1 b4)
Adding (at drone1 site2)
Adding (battery drone1 b2)
Updating (total-cost) (35) by 2 increase

Checking next happening (time 32)
Deleting (at drone1 site2)
Deleting (battery drone1 b2)
Adding (at drone1 ch2)
Adding (battery drone1 b1)
Updating (total-cost) (37) by 1 increase

Checking next happening (time 33)
Deleting (battery drone1 b1)
Adding (battery drone1 b6)
Updating (total-cost) (38) by 1 increase

Checking next happening (time 34)
Deleting (at drone1 ch2)
Deleting (battery drone1 b6)
Adding (at drone1 site5)
Adding (battery drone1 b4)
Updating (total-cost) (39) by 2 increase

Checking next happening (time 35)
Deleting (carrying drone1 water1)
Deleting (needs site5 water1)
Adding (empty drone1)
Adding (delivered water1 site5)
```

```
Adding (supply-at water1 site5)
Updating (total-cost) (41) by 1 increase
Plan executed successfully - checking goal
Plan valid
Final value: 42

Successful plans:
Value: 42
 plans/domain-A-medium-lama-first.plan 42
```

Val output of Problem large with Domain B and LAMA-first - domain-B-large-lama-first-val

```
Type-checking fly
...action passes type checking.
Type-checking inspect
...action passes type checking.
Type-checking pick-up-supply
...action passes type checking.
Type-checking deliver-supply
...action passes type checking.
Type-checking recharge
...action passes type checking.
Checking plan: plans/domain-B-large-lama-first.plan
Plan to validate:

Plan size: 52
1:
(pick-up-supply drone3 med2 depot_east)

2:
(pick-up-supply drone1 water1 depot_west)

3:
(fly drone1 depot_west ch_west b6 b1 b5)

4:
(fly drone1 ch_west site_w1 b5 b1 b4)

5:
(inspect drone1 site_w1)

6:
(fly drone1 site_w1 site_w2 b4 b1 b3)

7:
(fly drone1 site_w2 site_w3 b3 b1 b2)

8:
(deliver-supply drone1 water1 site_w3)

9:
(fly drone1 site_w3 site_w2 b2 b1 b1)

10:
(fly drone2 ch_east site_e1 b6 b1 b5)

11:
```

```
(inspect drone2 site_e1)

12:
(fly drone2 site_e1 ch_east b5 b1 b4)

13:
(fly drone2 ch_east site_e3 b4 b2 b2)

14:
(inspect drone2 site_e3)

15:
(fly drone2 site_e3 ch_east b2 b2 b0)

16:
(fly drone3 depot_east ch_east b6 b1 b5)

17:
(fly drone3 ch_east site_e2 b5 b1 b4)

18:
(deliver-supply drone3 med2 site_e2)

19:
(fly drone3 site_e2 site_e4 b4 b1 b3)

20:
(fly drone3 site_e4 ch_east b3 b2 b1)

21:
(recharge drone3 ch_east b1 b2)

22:
(fly drone3 ch_east site_e4 b2 b2 b0)

23:
(recharge drone2 ch_east b0 b1)

24:
(recharge drone2 ch_east b1 b2)

25:
(fly drone1 site_w2 ch_west b1 b1 b0)

26:
(recharge drone1 ch_west b0 b1)

27:
(fly drone1 ch_west site_w2 b1 b1 b0)

28:
(recharge drone2 ch_east b2 b3)

29:
(fly drone2 ch_east ch_west b3 b3 b0)

30:
```

```
(recharge drone2 ch_west b0 b1)

31:
(recharge drone2 ch_west b1 b2)

32:
(fly drone2 ch_west depot_west b2 b1 b1)

33:
(pick-up-supply drone2 med1 depot_west)

34:
(fly drone2 depot_west ch_west b1 b1 b0)

35:
(recharge drone2 ch_west b0 b1)

36:
(recharge drone2 ch_west b1 b2)

37:
(fly drone2 ch_west site_w2 b2 b1 b1)

38:
(deliver-supply drone2 med1 site_w2)

39:
(fly drone2 site_w2 ch_west b1 b1 b0)

40:
(recharge drone2 ch_west b0 b1)

41:
(recharge drone2 ch_west b1 b2)

42:
(recharge drone2 ch_west b2 b3)

43:
(fly drone2 ch_west ch_east b3 b3 b0)

44:
(recharge drone2 ch_east b0 b1)

45:
(recharge drone2 ch_east b1 b2)

46:
(fly drone2 ch_east depot_east b2 b1 b1)

47:
(pick-up-supply drone2 comm1 depot_east)

48:
(fly drone2 depot_east ch_east b1 b1 b0)

49:
```

```
(recharge drone2 ch_east b0 b1)

50:
(recharge drone2 ch_east b1 b2)

51:
(fly drone2 ch_east site_e4 b2 b2 b0)

52:
(deliver-supply drone2 comm1 site_e4)


Plan Validation details
-----------------------

Checking next happening (time 1)
Deleting (supply-at med2 depot_east)
Deleting (empty drone3)
Adding (carrying drone3 med2)
Updating (total-cost) (0) by 1 increase

Checking next happening (time 2)
Deleting (supply-at water1 depot_west)
Deleting (empty drone1)
Adding (carrying drone1 water1)
Updating (total-cost) (1) by 1 increase

Checking next happening (time 3)
Deleting (at drone1 depot_west)
Deleting (battery drone1 b6)
Adding (at drone1 ch_west)
Adding (battery drone1 b5)
Updating (total-cost) (2) by 1 increase

Checking next happening (time 4)
Deleting (at drone1 ch_west)
Deleting (battery drone1 b5)
Adding (at drone1 site_w1)
Adding (battery drone1 b4)
Updating (total-cost) (3) by 1 increase

Checking next happening (time 5)
Adding (inspected site_w1)
Updating (total-cost) (4) by 1 increase

Checking next happening (time 6)
Deleting (at drone1 site_w1)
Deleting (battery drone1 b4)
Adding (at drone1 site_w2)
Adding (battery drone1 b3)
Updating (total-cost) (5) by 1 increase

Checking next happening (time 7)
Deleting (at drone1 site_w2)
Deleting (battery drone1 b3)
Adding (at drone1 site_w3)
Adding (battery drone1 b2)
```

```
Updating (total-cost) (6) by 1 increase

Checking next happening (time 8)
Deleting (carrying drone1 water1)
Deleting (needs site_w3 water1)
Adding (empty drone1)
Adding (delivered water1 site_w3)
Adding (supply-at water1 site_w3)
Updating (total-cost) (7) by 1 increase

Checking next happening (time 9)
Deleting (at drone1 site_w3)
Deleting (battery drone1 b2)
Adding (at drone1 site_w2)
Adding (battery drone1 b1)
Updating (total-cost) (8) by 1 increase

Checking next happening (time 10)
Deleting (at drone2 ch_east)
Deleting (battery drone2 b6)
Adding (at drone2 site_e1)
Adding (battery drone2 b5)
Updating (total-cost) (9) by 1 increase

Checking next happening (time 11)
Adding (inspected site_e1)
Updating (total-cost) (10) by 1 increase

Checking next happening (time 12)
Deleting (at drone2 site_e1)
Deleting (battery drone2 b5)
Adding (at drone2 ch_east)
Adding (battery drone2 b4)
Updating (total-cost) (11) by 1 increase

Checking next happening (time 13)
Deleting (at drone2 ch_east)
Deleting (battery drone2 b4)
Adding (at drone2 site_e3)
Adding (battery drone2 b2)
Updating (total-cost) (12) by 2 increase

Checking next happening (time 14)
Adding (inspected site_e3)
Updating (total-cost) (14) by 1 increase

Checking next happening (time 15)
Deleting (at drone2 site_e3)
Deleting (battery drone2 b2)
Adding (at drone2 ch_east)
Adding (battery drone2 b0)
Updating (total-cost) (15) by 2 increase

Checking next happening (time 16)
Deleting (at drone3 depot_east)
Deleting (battery drone3 b6)
Adding (at drone3 ch_east)
```

```
Adding (battery drone3 b5)
Updating (total-cost) (17) by 1 increase

Checking next happening (time 17)
Deleting (at drone3 ch_east)
Deleting (battery drone3 b5)
Adding (at drone3 site_e2)
Adding (battery drone3 b4)
Updating (total-cost) (18) by 1 increase

Checking next happening (time 18)
Deleting (carrying drone3 med2)
Deleting (needs site_e2 med2)
Adding (empty drone3)
Adding (delivered med2 site_e2)
Adding (supply-at med2 site_e2)
Adding (accessible site_e4)
Updating (total-cost) (19) by 1 increase

Checking next happening (time 19)
Deleting (at drone3 site_e2)
Deleting (battery drone3 b4)
Adding (at drone3 site_e4)
Adding (battery drone3 b3)
Updating (total-cost) (20) by 1 increase

Checking next happening (time 20)
Deleting (at drone3 site_e4)
Deleting (battery drone3 b3)
Adding (at drone3 ch_east)
Adding (battery drone3 b1)
Updating (total-cost) (21) by 2 increase

Checking next happening (time 21)
Deleting (battery drone3 b1)
Adding (battery drone3 b2)
Updating (total-cost) (23) by 1 increase

Checking next happening (time 22)
Deleting (at drone3 ch_east)
Deleting (battery drone3 b2)
Adding (at drone3 site_e4)
Adding (battery drone3 b0)
Updating (total-cost) (24) by 2 increase

Checking next happening (time 23)
Deleting (battery drone2 b0)
Adding (battery drone2 b1)
Updating (total-cost) (26) by 1 increase

Checking next happening (time 24)
Deleting (battery drone2 b1)
Adding (battery drone2 b2)
Updating (total-cost) (27) by 1 increase

Checking next happening (time 25)
Deleting (at drone1 site_w2)
```

```
Deleting (battery drone1 b1)
Adding (at drone1 ch_west)
Adding (battery drone1 b0)
Updating (total-cost) (28) by 1 increase

Checking next happening (time 26)
Deleting (battery drone1 b0)
Adding (battery drone1 b1)
Updating (total-cost) (29) by 1 increase

Checking next happening (time 27)
Deleting (at drone1 ch_west)
Deleting (battery drone1 b1)
Adding (at drone1 site_w2)
Adding (battery drone1 b0)
Updating (total-cost) (30) by 1 increase

Checking next happening (time 28)
Deleting (battery drone2 b2)
Adding (battery drone2 b3)
Updating (total-cost) (31) by 1 increase

Checking next happening (time 29)
Deleting (at drone2 ch_east)
Deleting (battery drone2 b3)
Adding (at drone2 ch_west)
Adding (battery drone2 b0)
Updating (total-cost) (32) by 3 increase

Checking next happening (time 30)
Deleting (battery drone2 b0)
Adding (battery drone2 b1)
Updating (total-cost) (35) by 1 increase

Checking next happening (time 31)
Deleting (battery drone2 b1)
Adding (battery drone2 b2)
Updating (total-cost) (36) by 1 increase

Checking next happening (time 32)
Deleting (at drone2 ch_west)
Deleting (battery drone2 b2)
Adding (at drone2 depot_west)
Adding (battery drone2 b1)
Updating (total-cost) (37) by 1 increase

Checking next happening (time 33)
Deleting (supply-at med1 depot_west)
Deleting (empty drone2)
Adding (carrying drone2 med1)
Updating (total-cost) (38) by 1 increase

Checking next happening (time 34)
Deleting (at drone2 depot_west)
Deleting (battery drone2 b1)
Adding (at drone2 ch_west)
Adding (battery drone2 b0)
```

```
Updating (total-cost) (39) by 1 increase

Checking next happening (time 35)
Deleting (battery drone2 b0)
Adding (battery drone2 b1)
Updating (total-cost) (40) by 1 increase

Checking next happening (time 36)
Deleting (battery drone2 b1)
Adding (battery drone2 b2)
Updating (total-cost) (41) by 1 increase

Checking next happening (time 37)
Deleting (at drone2 ch_west)
Deleting (battery drone2 b2)
Adding (at drone2 site_w2)
Adding (battery drone2 b1)
Updating (total-cost) (42) by 1 increase

Checking next happening (time 38)
Deleting (carrying drone2 med1)
Deleting (needs site_w2 med1)
Adding (empty drone2)
Adding (delivered med1 site_w2)
Adding (supply-at med1 site_w2)
Updating (total-cost) (43) by 1 increase

Checking next happening (time 39)
Deleting (at drone2 site_w2)
Deleting (battery drone2 b1)
Adding (at drone2 ch_west)
Adding (battery drone2 b0)
Updating (total-cost) (44) by 1 increase

Checking next happening (time 40)
Deleting (battery drone2 b0)
Adding (battery drone2 b1)
Updating (total-cost) (45) by 1 increase

Checking next happening (time 41)
Deleting (battery drone2 b1)
Adding (battery drone2 b2)
Updating (total-cost) (46) by 1 increase

Checking next happening (time 42)
Deleting (battery drone2 b2)
Adding (battery drone2 b3)
Updating (total-cost) (47) by 1 increase

Checking next happening (time 43)
Deleting (at drone2 ch_west)
Deleting (battery drone2 b3)
Adding (at drone2 ch_east)
Adding (battery drone2 b0)
Updating (total-cost) (48) by 3 increase

Checking next happening (time 44)
```

```
Deleting (battery drone2 b0)
Adding (battery drone2 b1)
Updating (total-cost) (51) by 1 increase

Checking next happening (time 45)
Deleting (battery drone2 b1)
Adding (battery drone2 b2)
Updating (total-cost) (52) by 1 increase

Checking next happening (time 46)
Deleting (at drone2 ch_east)
Deleting (battery drone2 b2)
Adding (at drone2 depot_east)
Adding (battery drone2 b1)
Updating (total-cost) (53) by 1 increase

Checking next happening (time 47)
Deleting (supply-at comm1 depot_east)
Deleting (empty drone2)
Adding (carrying drone2 comm1)
Updating (total-cost) (54) by 1 increase

Checking next happening (time 48)
Deleting (at drone2 depot_east)
Deleting (battery drone2 b1)
Adding (at drone2 ch_east)
Adding (battery drone2 b0)
Updating (total-cost) (55) by 1 increase

Checking next happening (time 49)
Deleting (battery drone2 b0)
Adding (battery drone2 b1)
Updating (total-cost) (56) by 1 increase

Checking next happening (time 50)
Deleting (battery drone2 b1)
Adding (battery drone2 b2)
Updating (total-cost) (57) by 1 increase

Checking next happening (time 51)
Deleting (at drone2 ch_east)
Deleting (battery drone2 b2)
Adding (at drone2 site_e4)
Adding (battery drone2 b0)
Updating (total-cost) (58) by 2 increase

Checking next happening (time 52)
Deleting (carrying drone2 comm1)
Deleting (needs site_e4 comm1)
Adding (empty drone2)
Adding (delivered comm1 site_e4)
Adding (supply-at comm1 site_e4)
Updating (total-cost) (60) by 1 increase
Plan executed successfully - checking goal
Plan valid
Final value: 61
```

```
Successful plans:
Value: 61
 plans/domain-B-large-lama-first.plan 61
```

Use of AI

AI tools were used as a support throughout this project, specifically, ChatGPT was used to find a way to automate the extraction of results from the plan files, the Fast Downward output into a .csv file, ChatGPT then suggested outputting the data into structured directories (my-plans, my-results and my-validation) and provided a Python script to parse the values for our Results Table data (*Instance, Domain Model, Planner, Solved, Runtime (s), Plan Length, Plan Cost, Expanded states*) from these newly outputted files.

Gemini Notebook was used as a collaborative analytical and technical editing assistant. Specifically, the AI assisted in: (1) analyzing state-space bounds and verifying PDDL action schemata and predicate structures, (2) clarifying search theory concepts including the mechanics of A* with LM-cut versus greedy search with LAMA-first, (3) parsing raw Fast Downward benchmark CSV data to compute relative runtime and state expansion metrics, and (4) turn PDDL code into tables, as in Figure 1.

References

Bit-Monnot, A., Bailon-Ruiz, R., & Lacroix, S. (2018). A Local Search Approach to Observation Planning with Multiple UAVs. *Proceedings of the International Conference on Automated Planning and Scheduling*, 28(1), 437–445. https://doi.org/10.1609/icaps.v28i1.13924

Helmert, M. (2006). The Fast Downward planning system. *Journal of Artificial Intelligence Research*, *26*, 191–246. https://doi.org/10.1613/jair.1705

Hoffmann, J. (2003). The Metric-FF planning system: Translating "ignoring delete lists" to numeric state variables. *Journal of Artificial Intelligence Research*, *20*, 291–341.

Howey, R., Long, D., & Fox, M. (2004). Val: Automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *16th IEEE International Conference on Tools with Artificial Intelligence* (pp. 294–301). https://doi.org/10.1109/ictai.2004.120

Mersheeva, V., & Friedrich, G. (2015). Multi-UAV monitoring with priorities and limited energy resources. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, *25*(1), 347–355. https://doi.org/10.1609/icaps.v25i1.13695